Computer Vision

T1 Classical Computer Vision

University Master's Degree in
Machine Learning

upna

Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Classical Computer Vision

- Introduction
  - Early computer vision needed handcrafted feature engineering
  - Deep learning later automated this process

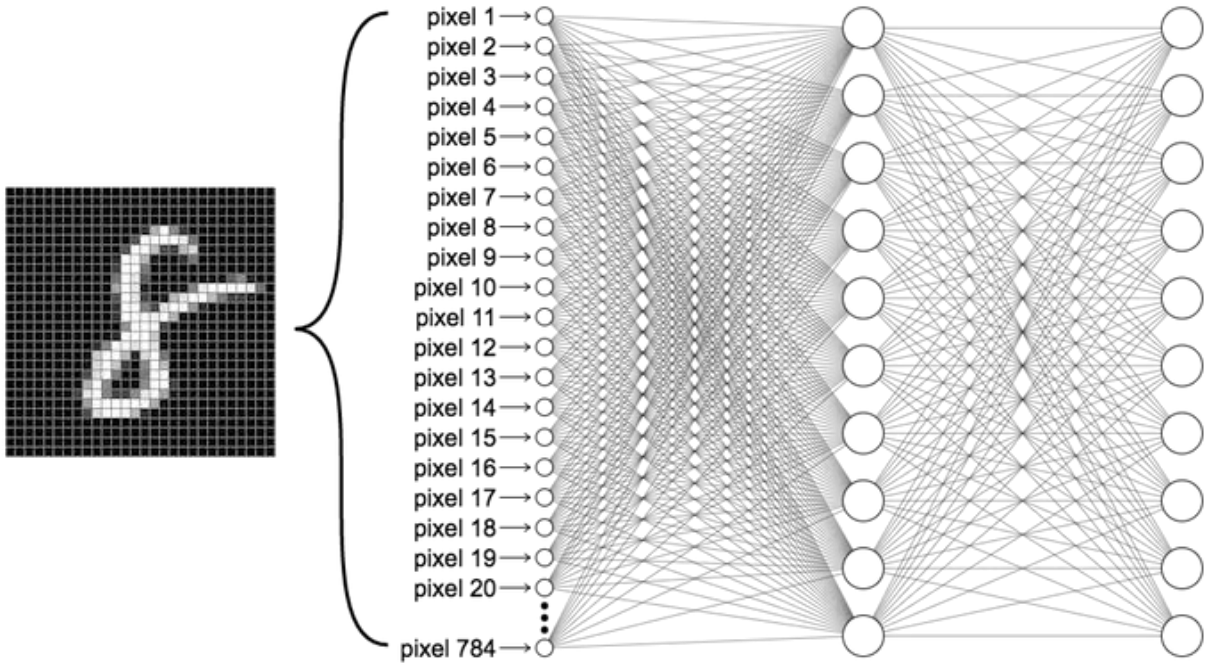
  - Image features and descriptors
  - Classical feature extraction techniques (HoG, SIFT)
  - Feature-based pipelines with machine learning
  - Transition to deep learning

Classical Computer Vision

- From Pixels to Features
  - Why not use raw pixels?
    - High dimensionality, lack of invariance (translation, scale, rotation).
  - What is a feature?
    - Compact, informative representation of an image (edges, corners, textures, etc.).
  - Low-level features examples:
    - Color histograms, edge detectors (Sobel, Canny).
    - Texture descriptors (Gabor filters, LBP).
  - Goal: obtain a vector representation for machine learning models.

Classical Computer Vision

- Direct approach: Raw pixel intensities as features
  - Each image can be represented as a vector of pixel intensities:
    - Example: a 64×64 grayscale image → 4096-dimensional vector.
    - For RGB images: multiply by 3 channels.
  - Feed directly into classifiers (e.g., logistic regression, SVM, k-NN).



Classical Computer Vision

- Direct approach: Raw pixel intensities as features
  - Advantages:
    - Simple, no preprocessing or feature extraction.
    - Works for very simple datasets (e.g., handwritten digits like MNIST).
  - Limitations:
    - Very sensitive to translation, rotation, and scale.
    - No invariance: same object shifted a few pixels → different feature vector.
    - Curse of dimensionality: large number of features → overfitting.
    - Poor generalization across lighting conditions or backgrounds.

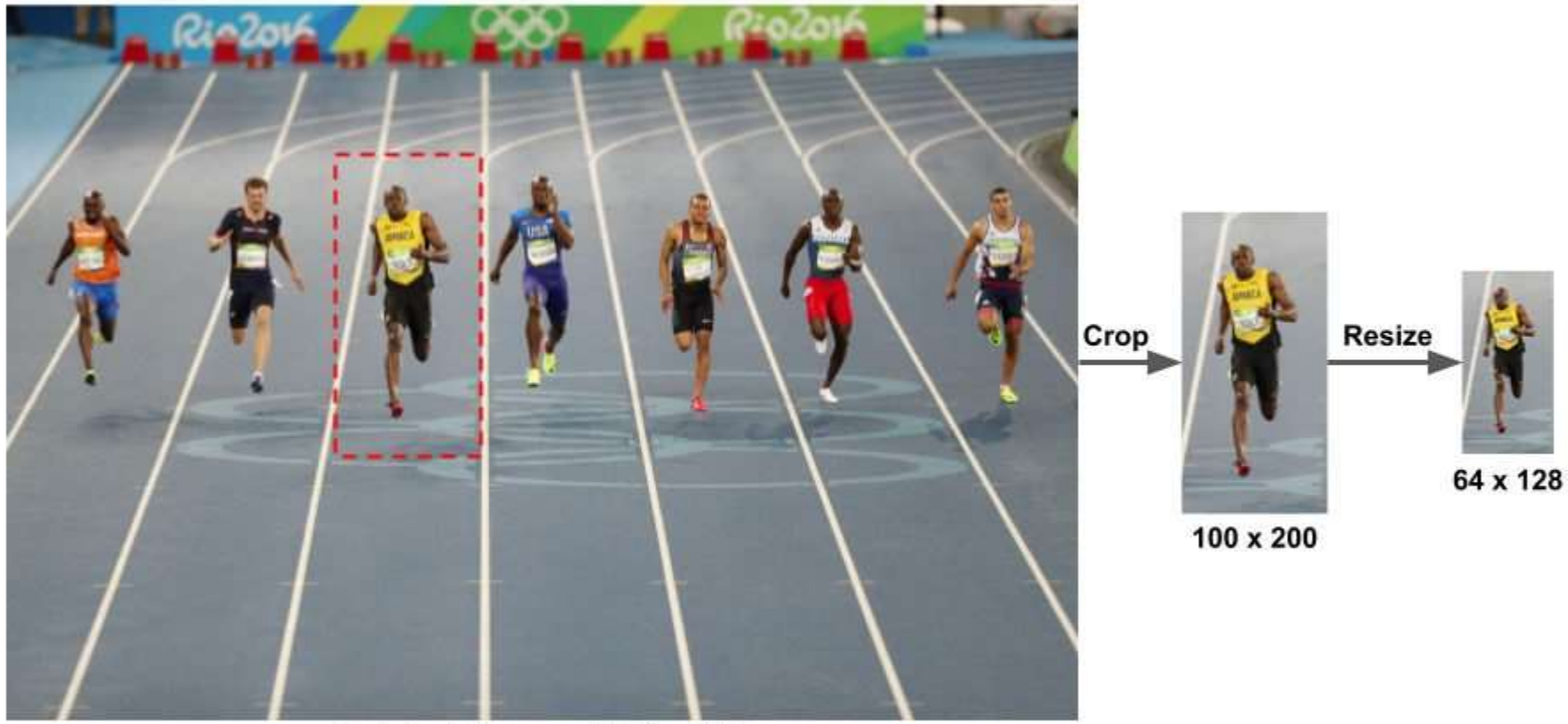
  - To overcome these issues, researchers began designing features that captured structure and invariance like HoG and SIFT

Classical Computer Vision

- Histogram of Oriented Gradients (HoG)
  - Motivation: robust detection of shapes and objects (especially people).
  - Key idea: local distribution of gradient orientations captures object structure.
  - Pipeline:
    1. Compute image gradients (x and y directions).
    2. Divide image into cells (e.g., 8×8 pixels).
    3. Create histograms of gradient orientations for each cell.
    4. Normalize histograms in blocks for illumination invariance.
    5. Concatenate all histograms → feature vector.

Classical Computer Vision

- Histogram of Oriented Gradients (HoG)
  - Pipeline:
    1. Compute image gradients (x and y directions).



Original Image : 720 x 475

Classical Computer Vision

- Histogram of Oriented Gradients (HoG)
  - Pipeline:
    1. Compute image gradients (x and y directions).

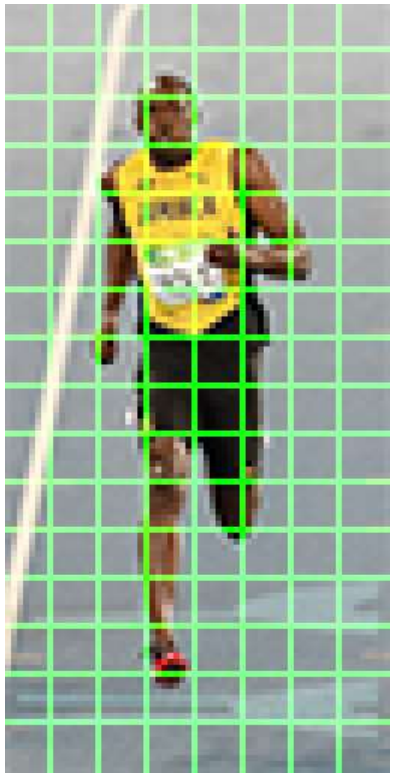
-1	0	1

-1
0
1

Left : Absolute value of x-gradient. Center : Absolute value of y-gradient.
Right : Magnitude of gradient.

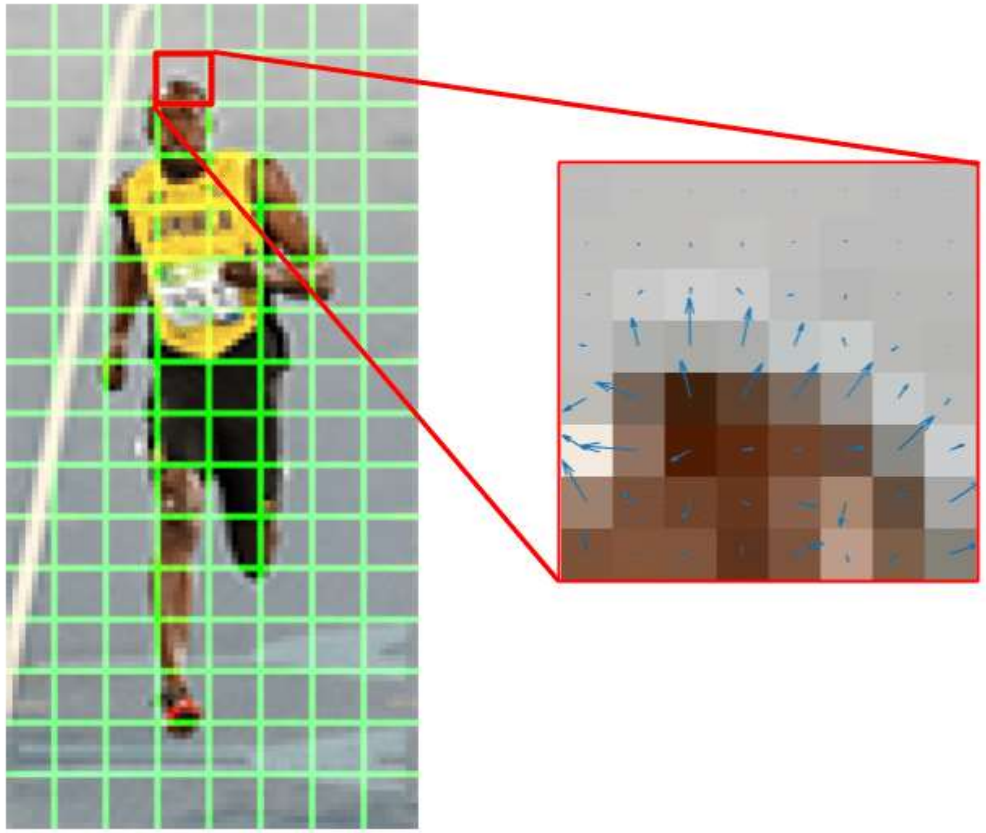
Classical Computer Vision

- Histogram of Oriented Gradients (HoG)
  - Pipeline:
    1. Divide image into cells (e.g., 8×8 pixels).



Classical Computer Vision

- Histogram of Oriented Gradients (HoG)
  - Pipeline:
    1. Create histograms of gradient orientations for each cell.



2	3	4	4	3	4	2	2
5	11	17	13	7	9	3	4
11	21	23	27	22	17	4	6
23	99	165	135	85	32	26	2
91	155	133	136	144	152	57	28
98	196	76	38	26	60	170	51
165	60	60	27	77	85	43	136
71	13	34	23	108	27	48	110

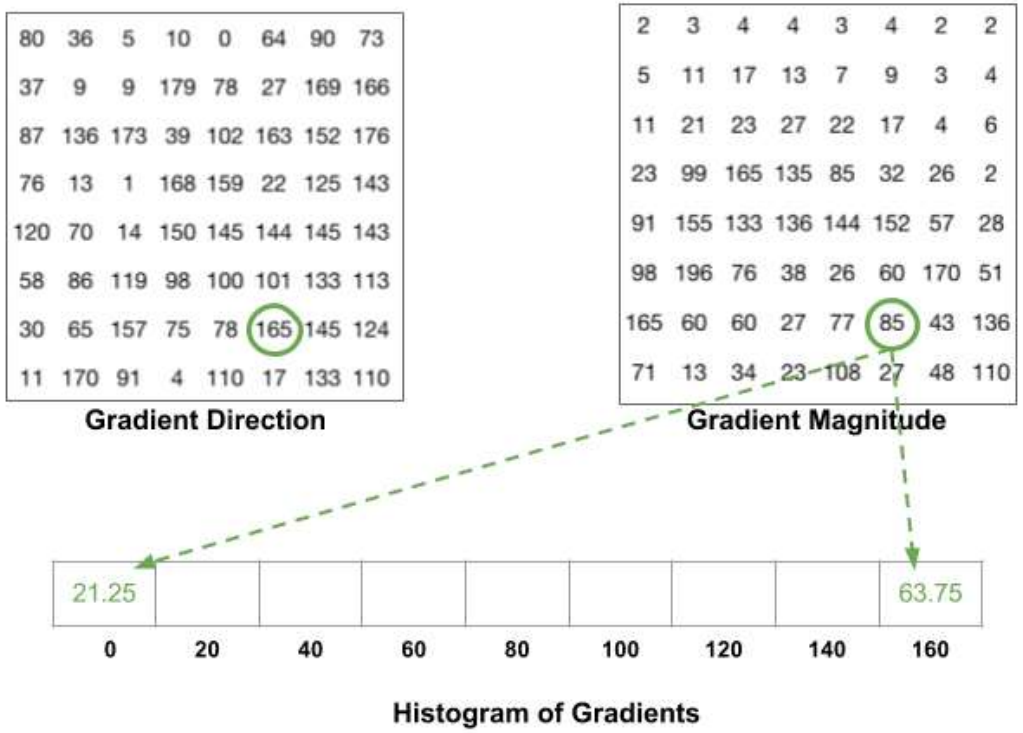
Gradient Magnitude

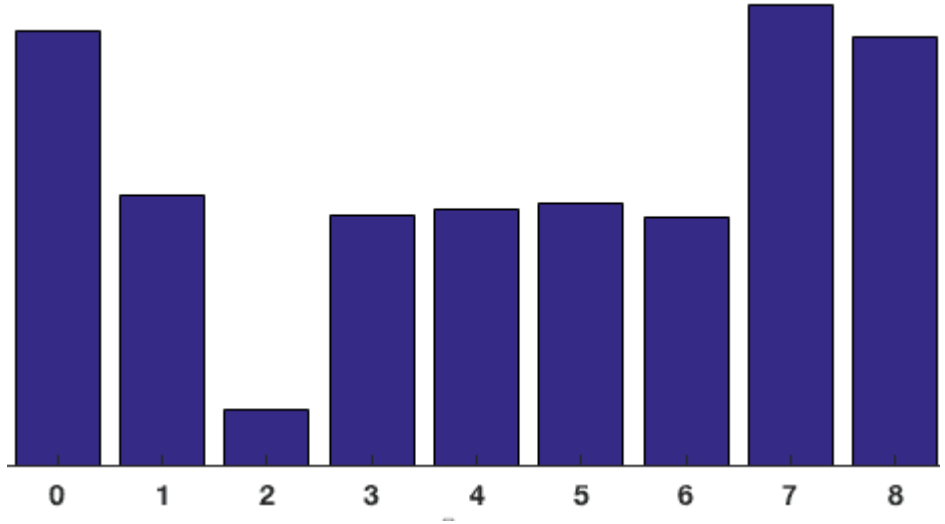
80	36	5	10	0	64	90	73
37	9	9	179	78	27	169	166
87	136	173	39	102	163	152	176
76	13	1	168	159	22	125	143
120	70	14	150	145	144	145	143
58	86	119	98	100	101	133	113
30	65	157	75	78	165	145	124
11	170	91	4	110	17	133	110

Gradient Direction

Classical Computer Vision

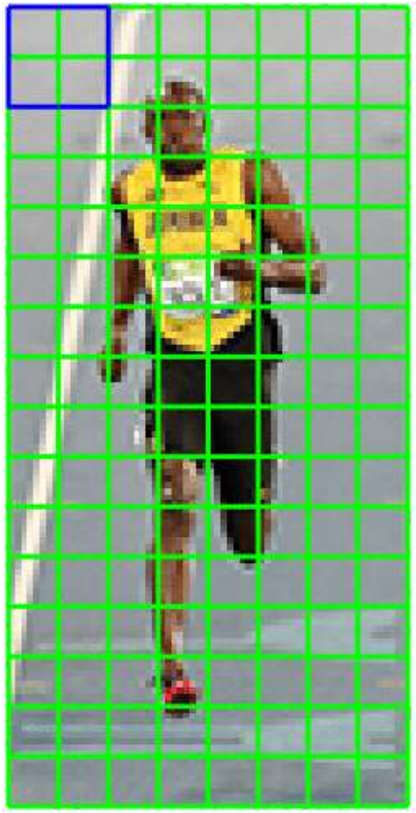
- Histogram of Oriented Gradients (HoG)
  - Pipeline:
    1. Create histograms of gradient orientations for each cell.





Classical Computer Vision

- Histogram of Oriented Gradients (HoG)
  - Pipeline:
    1. Normalize histograms in blocks for illumination invariance.



Classical Computer Vision

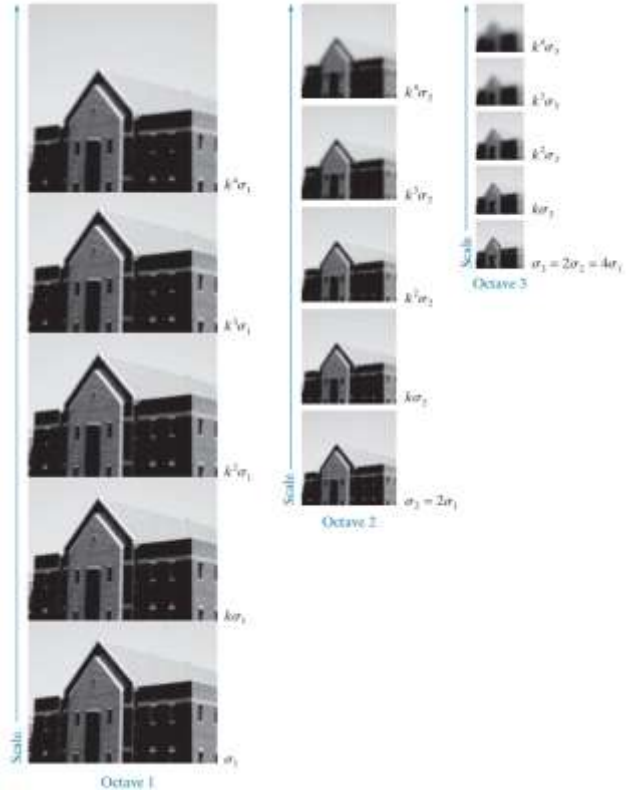
- Histogram of Oriented Gradients (HoG)
  - Pipeline:
    1. Concatenate all histograms → feature vector.

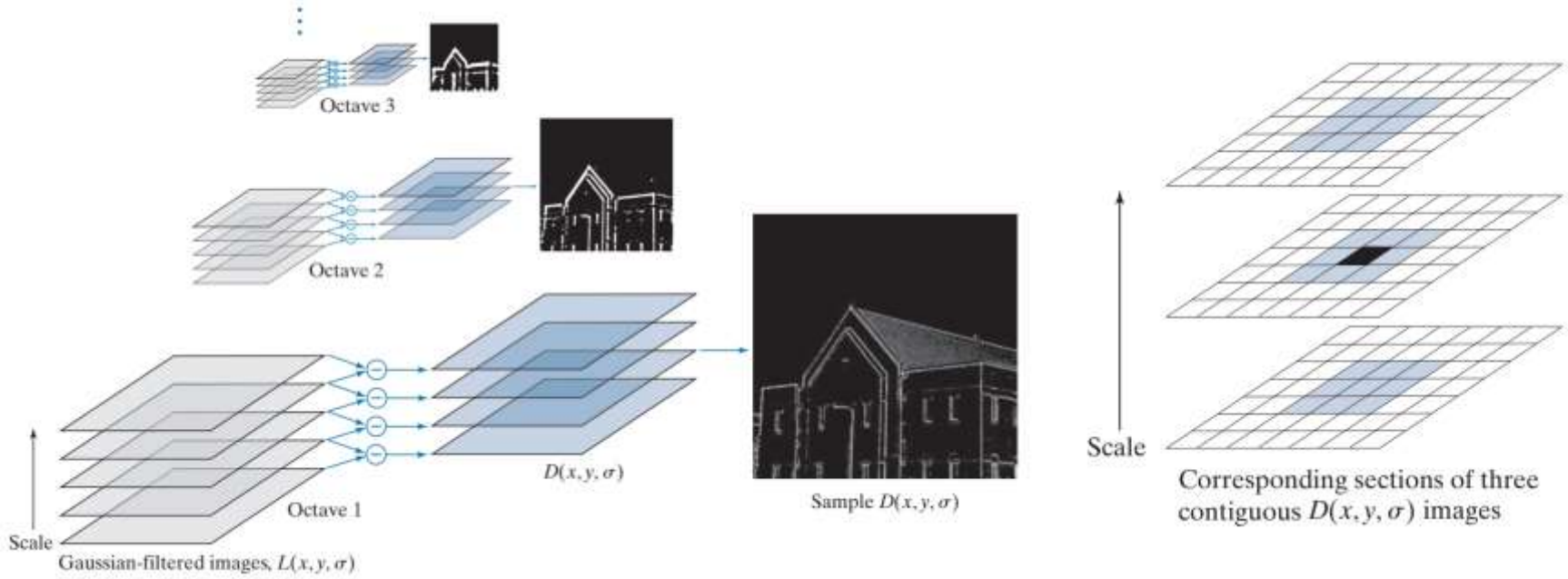
Classical Computer Vision

- Scale-Invariant Feature Transform (SIFT)
  - Motivation: detect and describe local features invariant to scale and rotation.
  - Pipeline:
    1. Keypoint detection: find extrema in scale space (using Difference of Gaussians).
    2. Keypoint localization: discard low-contrast or edge responses.
    3. Orientation assignment: compute dominant gradient direction per keypoint.
    4. Descriptor computation: build a 128-D vector from gradient histograms around the keypoint.
  - Properties:
    - Invariant to rotation, scale, moderate illumination changes.
    - Useful for image matching, object recognition, stitching.

Classical Computer Vision

- Scale-Invariant Feature Transform (SIFT)
  - Pipeline:
    1. Keypoint detection: find extrema in scale space (using Difference of Gaussians).





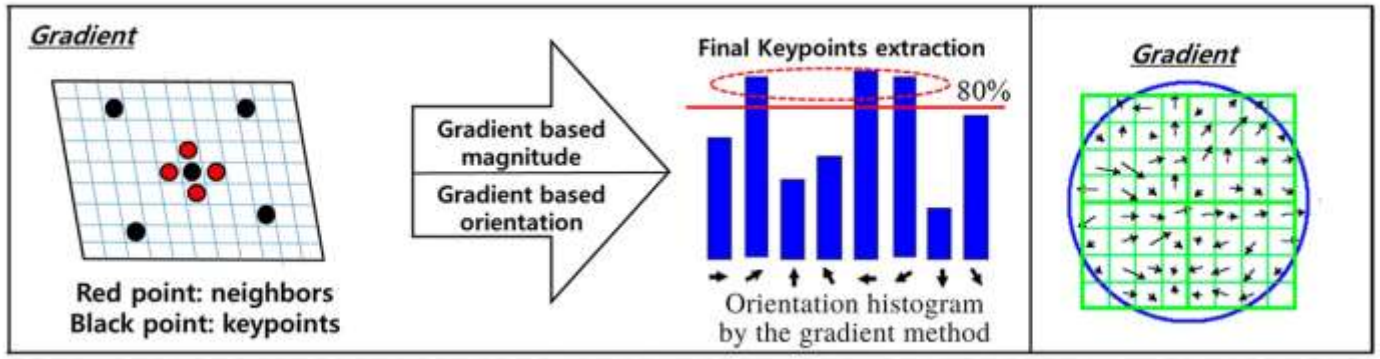
Classical Computer Vision

- Scale-Invariant Feature Transform (SIFT)
  - Pipeline:
    1. Keypoint localization: discard low-contrast or edge responses.

(a) shows detected keypoints at different scales. The SIFT algorithm removes keypoints that have low contrast (the remaining keypoints are shown in the second image (b)). Then, it further removes keypoints that are located on edges. The final set of keypoints is shown in the last image (c).

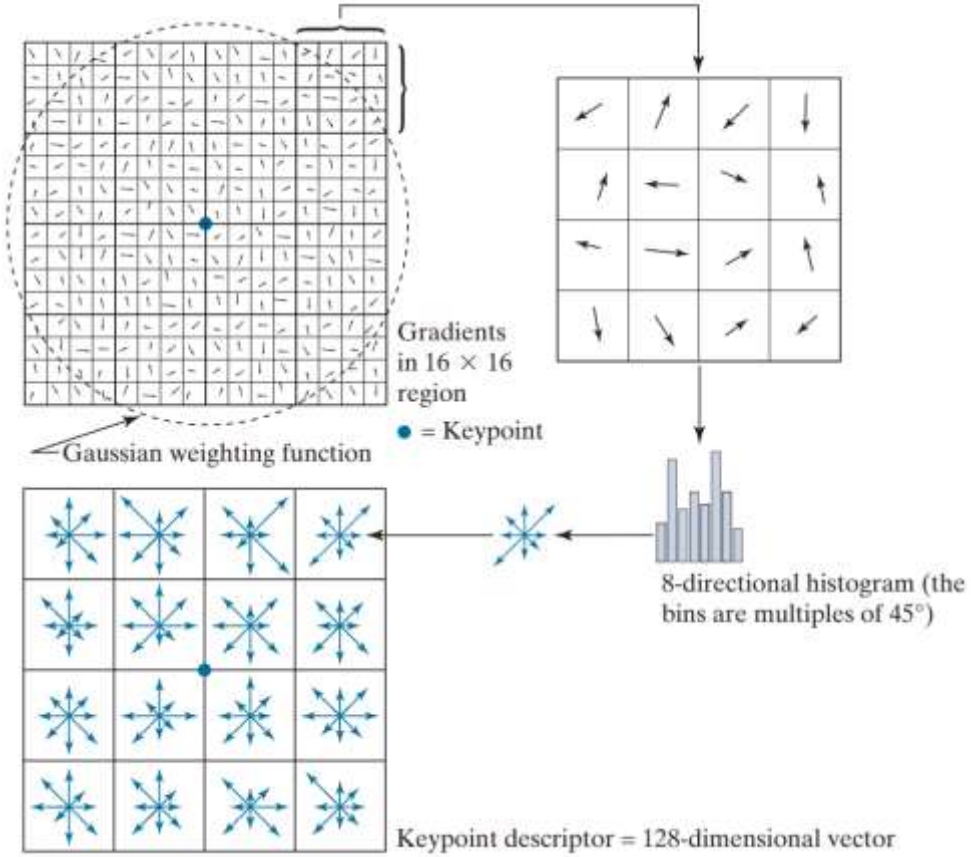
Classical Computer Vision

- Scale-Invariant Feature Transform (SIFT)
  - Pipeline:
    1. Orientation assignment: compute dominant gradient direction per keypoint.



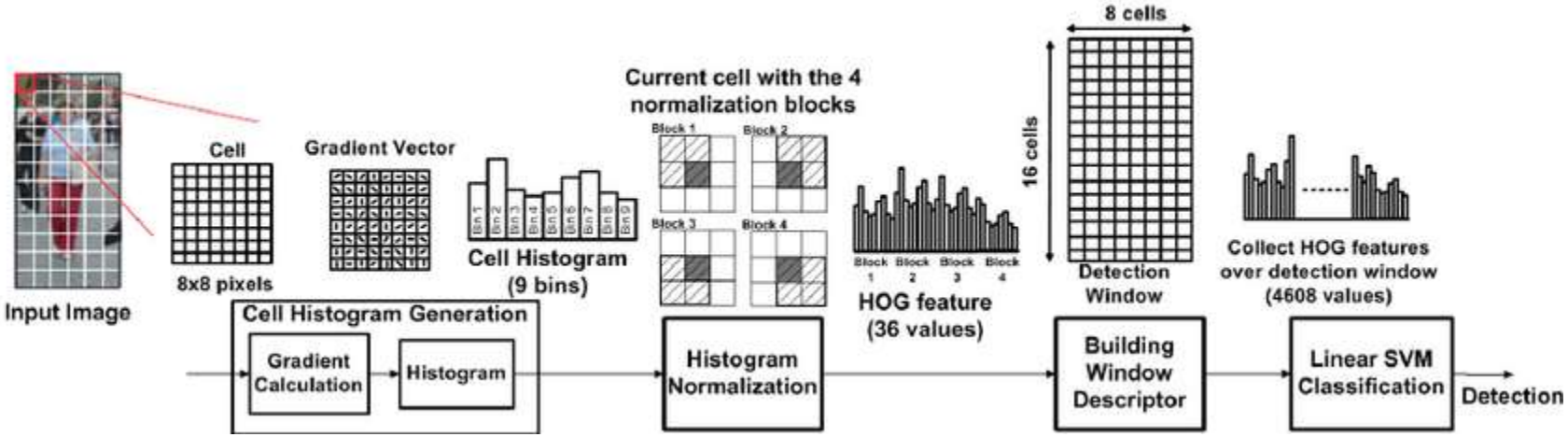
Classical Computer Vision

- Scale-Invariant Feature Transform (SIFT)
  - Pipeline:
    1. Descriptor computation: build a 128-D vector from gradient histograms around the keypoint.



Classical Computer Vision

- Feature-Based Pipelines
  - Typical workflow:
    - Feature extraction (HoG, SIFT, LBP, etc.)
    - Feature encoding (e.g., Bag of Visual Words, Fisher Vectors)
    - Classification or detection using ML models:
      - SVMs (e.g., HoG + linear SVM for people detection)
      - Random Forests
      - k-NN



Classical Computer Vision

- Transition to Deep Learning
  - Typical workflow has some limitations:
    - Heavy manual engineering.
    - Poor generalization to complex scenes.
    - Limited representation power.
  - Whats next?
    - Convolutional Neural Networks (CNNs) learn hierarchical features automatically.
    - Early CNN layers resemble Gabor filters or HoG-like gradient detectors.